KAPASA MAKASA UNIVERSITY

SCHOOL OF APPLIED SCIENCES AND EDUCATION

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

S/N	Student Name	Student Number
1	Salifyanji Mutambo	20231585
2	Frazer Ntombi	20232013
3	Chimba Chanda	20232011
4	Chipego Shambozee	22175284
5	Alinet Hangoba	20231685
6	Chanda Mercy	22180165
7	Lawrence Tembo	20232016
8	Esther Mbewe	20230496
9	Cuthbelt Ndhlovu	20231784
10	Humphrey Chileshe	20231381
11	Alinaswe Simfukwe	20231405

GROUP ASSESMENT LAB REPORT FOR CYS 415 MOBILE FORENSICS

BY
GROUP 7

LECTURER: MR. DARRY DERRICK

LAB REPORT: ANDROID SECURITY ASSESSMENT

1. Introduction and Objectives

This report documents the practical execution of Lab 5, which focuses on identifying and bypassing SSL certificate pinning on an Android application running inside a controlled Android Virtual Device (AVD). The purpose of the exercise is to understand how mobile applications protect network traffic using certificate pinning, how that protection can be circumvented for authorised security testing, and what kinds of vulnerabilities become visible once traffic can be intercepted.

The lab was carried out entirely on a Kali Linux host running an Android Studio emulator, with Burp Suite used as the intercepting proxy. Root access on the AVD was obtained using rootAVD and Magisk, the LSPosed framework was installed to support Xposed-style modules, and the SSL Killer module was used as the primary bypass mechanism. Where the module-based bypass would not be sufficient (for example, against native or non-standard TLS stacks), the lab guide also outlines a Frida/Objection fallback, which is described in Section 10 for completeness.

Every action below was performed against the student's own emulator and a sample/test application, consistent with the authorised-testing scope of the module. No real user data or production services were targeted at any point.

2. Tools and Environment Summary

Component	Details
Host OS	Kali Linux (bare metal)
Target device	Android Studio AVD – Pixel, Google APIs (non-Play) image
Root method	rootAVD + Magisk
Hooking framework	LSPosed (Zygisk mode)
Pinning bypass module	SSL Killer v2.0 Beta
Intercepting proxy	Burp Suite Community Edition
Fallback tooling	Frida + Objection

3. Part 0 & Part A – Launching and Rooting the AVD

The first practical step was to bring the virtual device up and confirm that ADB could see it before touching anything else. Once the Pixel AVD had fully booted, adb devices was run to confirm the connection, which returned emulator-5554 as an attached device — the green light to move on to rooting.

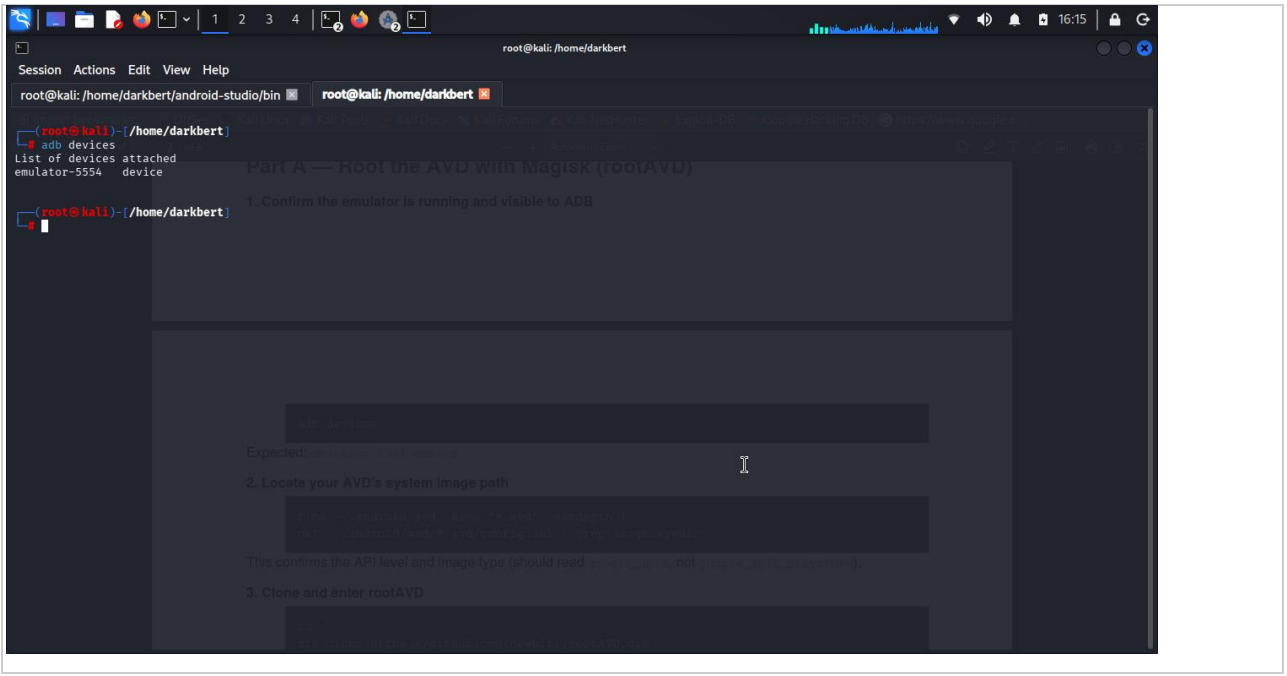


Figure 1. adb devices confirming the AVD (emulator-5554) is attached and ready before rooting begins.

With the device confirmed, the AVD's system image path was located to verify it was a non-Play "Google APIs" image (a requirement for rootAVD to work), and rootAVD.sh was cloned and executed. Running the script with no arguments causes it to auto-detect the running AVD and system image and present a numbered menu; the entry matching the Pixel AVD's API level was selected. The screenshots below show the script working through its patching sequence — repacking the ramdisk, injecting the Magisk init binary, and pushing the patched image back to the device.

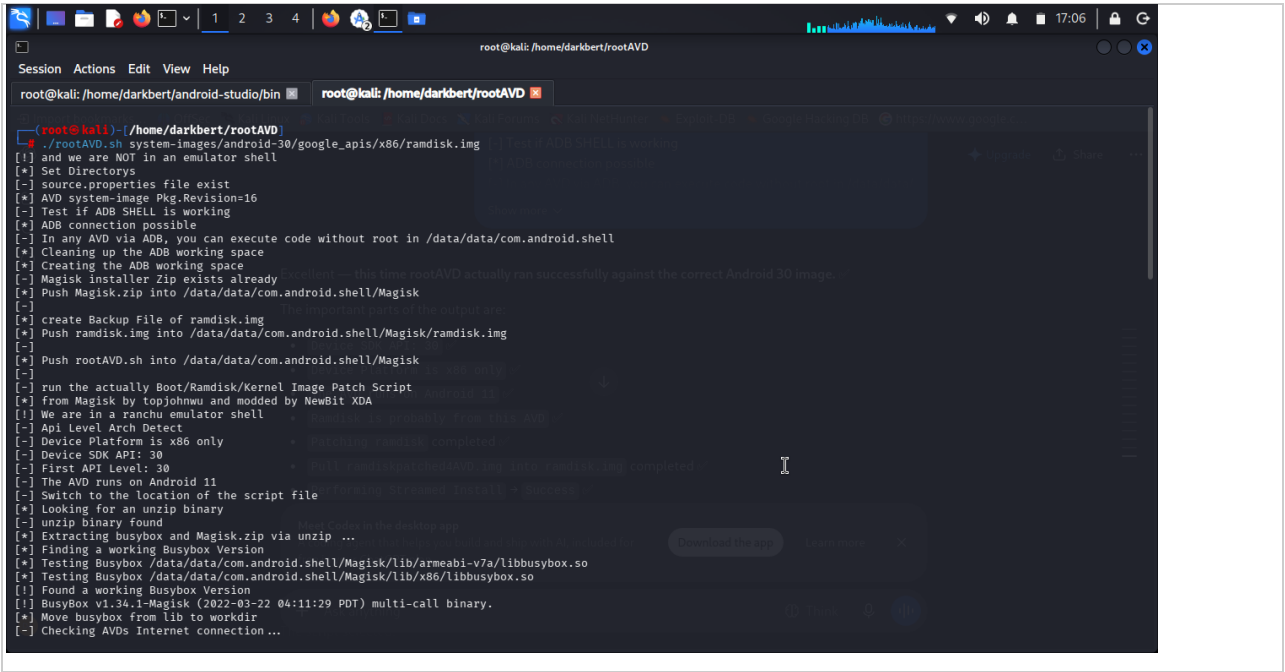


Figure 2. rootAVD.sh detecting the AVD and beginning the Magisk boot/ramdisk patching sequence.

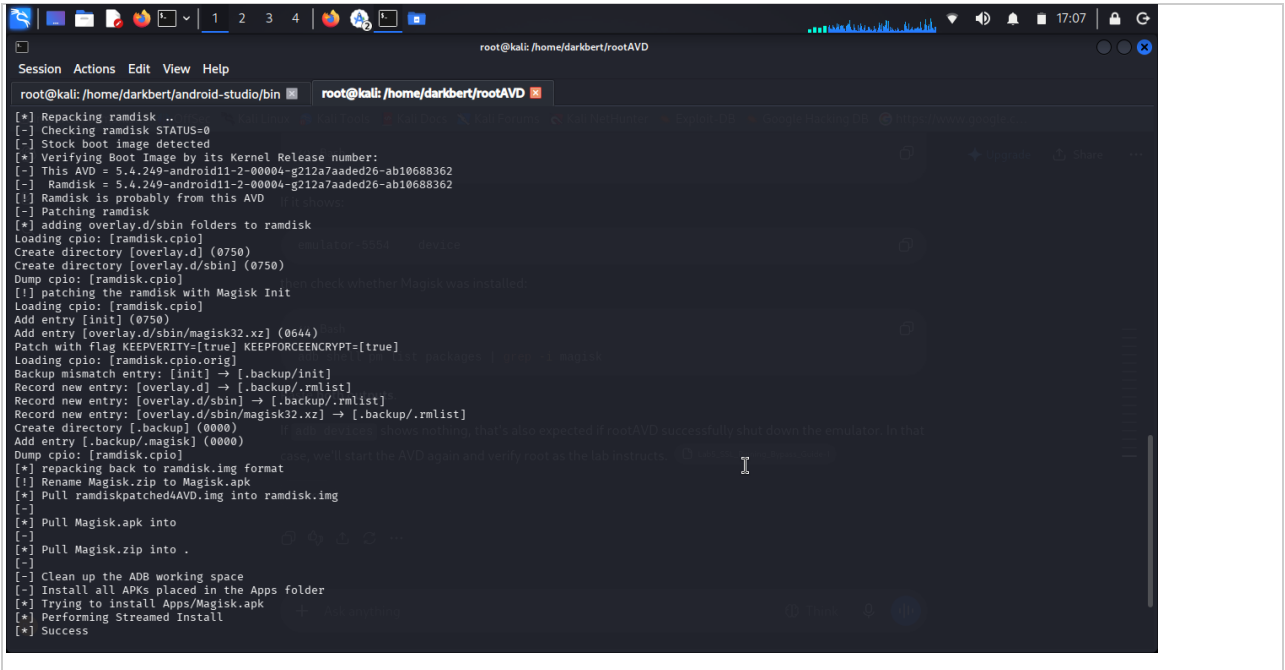


Figure 3. rootAVD.sh repacking the ramdisk and installing the Magisk APK via a streamed install.

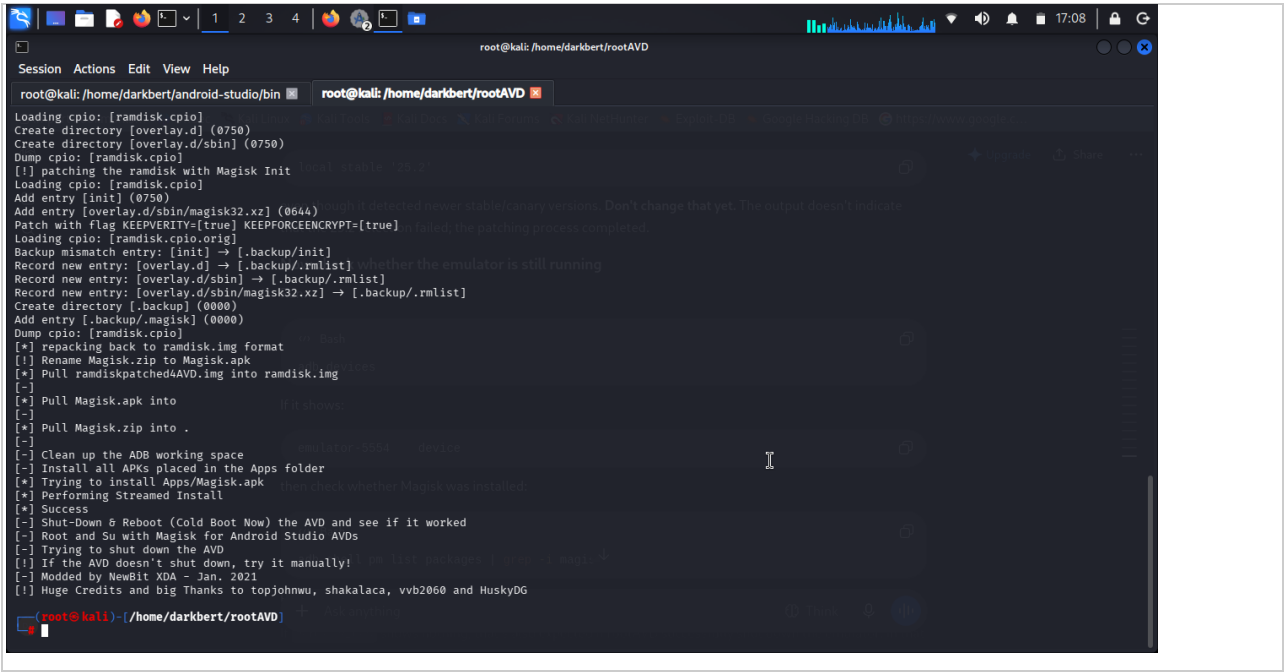


Figure 4. rootAVD.sh completing the patch process and shutting the AVD down ready for a cold boot.

After the script finished and the emulator rebooted, the Magisk app's on-device setup was completed (accepting the "Additional Setup Required" prompt), and the device came back up to a normal home screen confirmation that the patched boot image loaded correctly and that Magisk was ready to grant root on request.

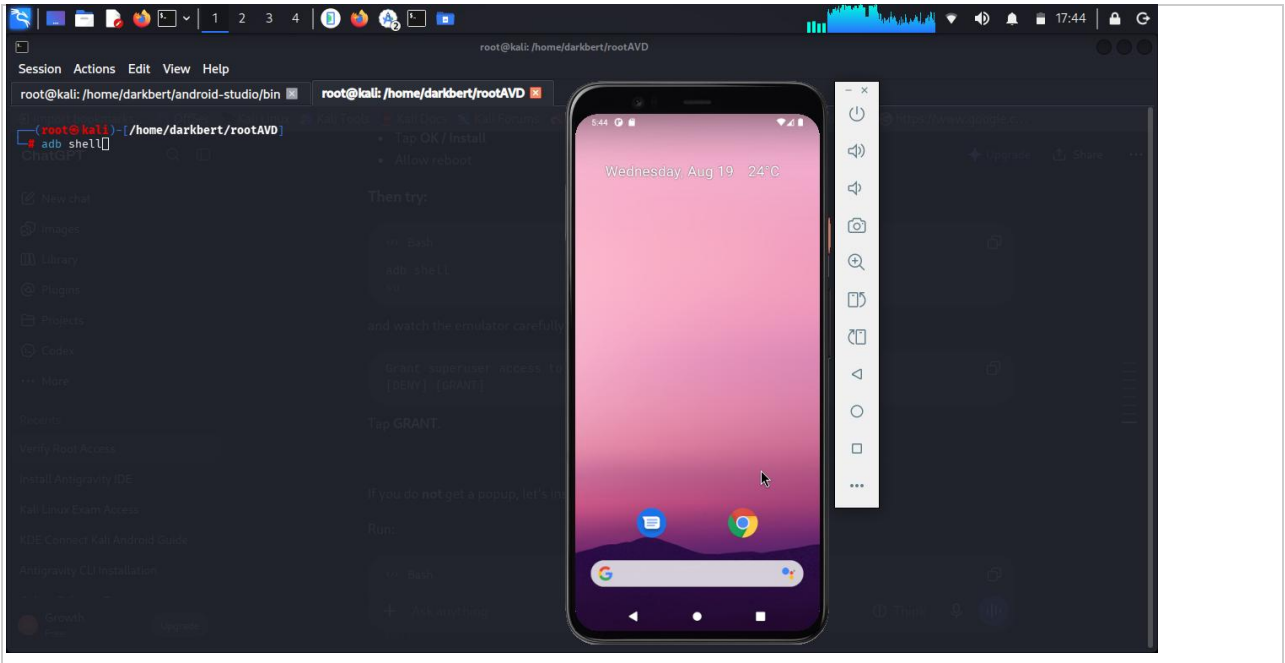


Figure 5. The AVD home screen after the Magisk-patched boot completed successfully.

Root access itself was verified from the Kali terminal with adb shell followed by su, approving the resulting superuser prompt on the emulator screen. A # prompt in place of the usual $ confirmed that root had been granted, closing out Part A.

4. Part B – Configuring the Burp Suite Proxy Listener

Burp Suite was launched with a temporary project and default settings. Under Proxy → Proxy settings, the default listener on 127.0.0.1:8080 was edited so that it bound to All interfaces rather than just localhost this is what allows the emulator, which reaches the host over a virtual network interface rather than true localhost, to actually connect to the listener. Burp was then left running in the background for the remainder of the lab. No dedicated screenshot was taken for this configuration step, since the effect of it is confirmed directly in Part D and Part C, once the emulator successfully reaches the listener at 10.0.2.2:8080.

5. Part C – Installing Burp's CA Certificate on the Emulator

For Burp to transparently intercept HTTPS traffic, its CA certificate first had to be trusted by the Android system. From the emulator's browser, http://10.0.2.2:8080 was opened — 10.0.2.2 being the emulator's alias for the host machine — and the CA Certificate link on Burp's welcome page was used to download cacert.der to the device's Downloads folder.

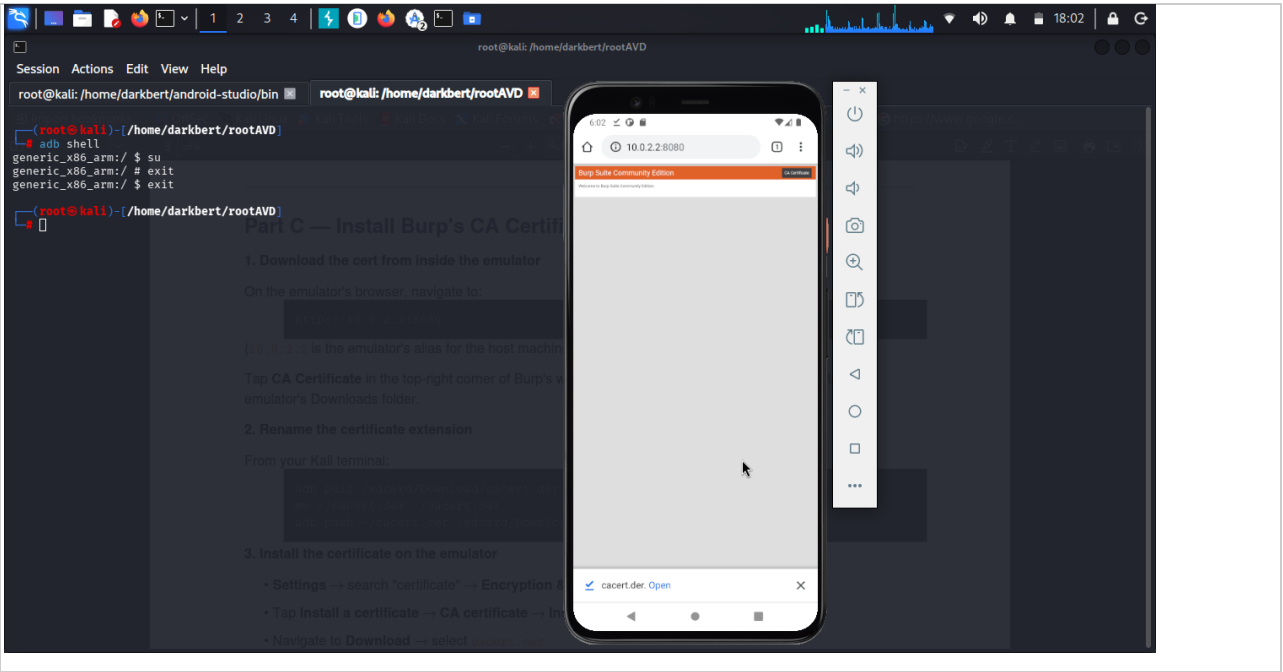


Figure 6. Navigating to Burp's built-in certificate page at 10.0.2.2:8080 from inside the emulator and downloading cacert.der.

Android will not offer a .der file for installation as a CA certificate through the Settings UI, so the file was pulled back to the host, renamed to a .cer extension, and pushed back onto the device:

```
adb pull /sdcard/Download/cacert.der ~/cacert.der
mv ~/cacert.der ~/cacert.cer
adb push ~/cacert.cer /sdcard/Download/cacert.cer
```

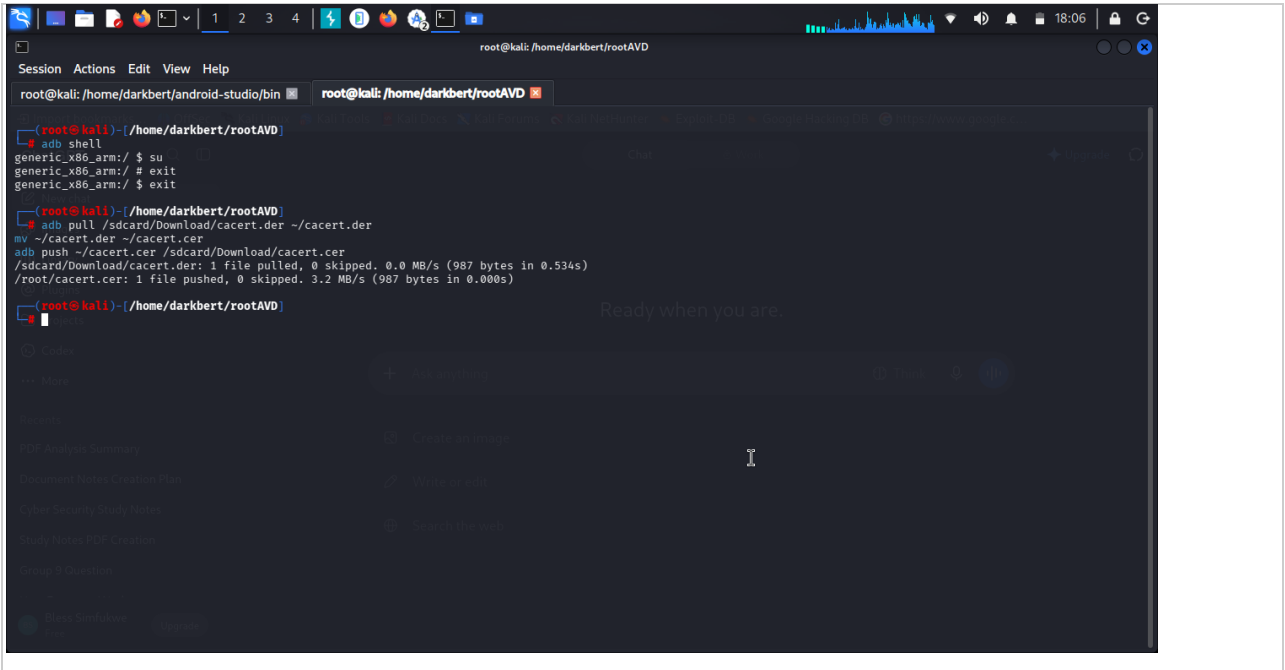


Figure 7. Renaming the pulled certificate to a .cer extension and pushing it back to the device's Download folder.

The renamed certificate was then installed through Settings → Encryption & credentials → Install a certificate → CA certificate, confirming past Android's "Install anyway" warning. Once installed, the device reported "CA certificate installed", meaning the emulator would now trust Burp as a certificate authority for the interception that followed.

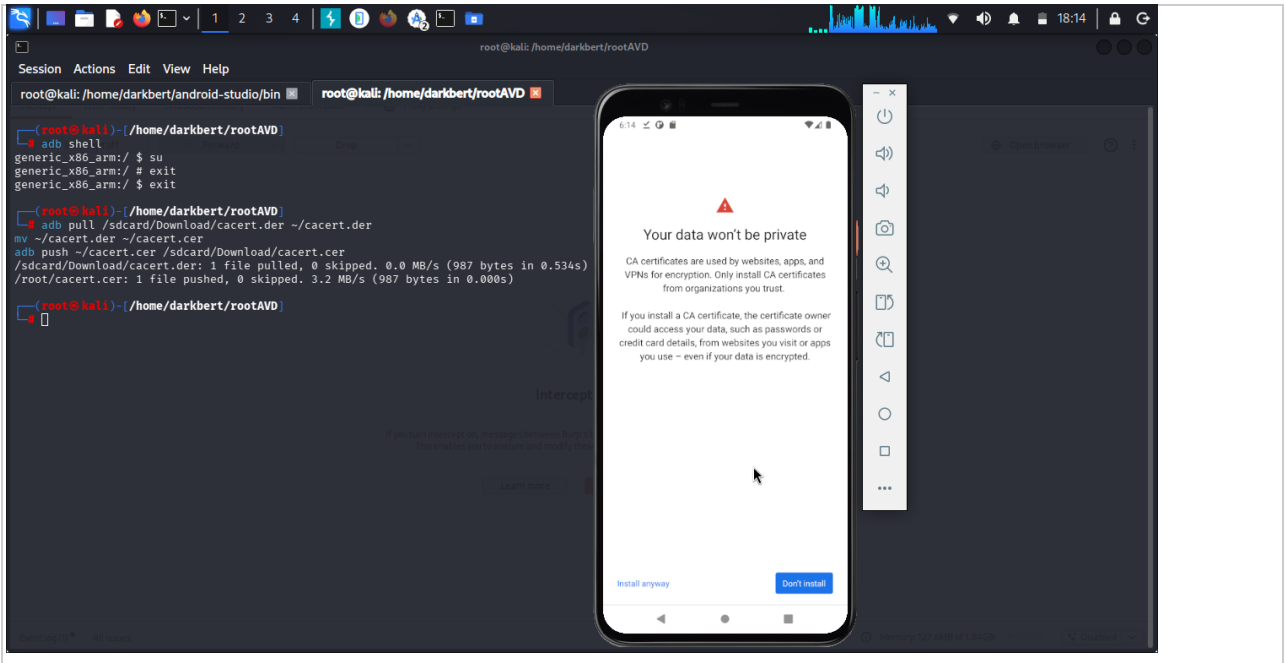


Figure 8. Confirming the "Install anyway" warning while installing Burp's CA certificate as a trusted authority.

6. Part D – Routing the Emulator's Traffic Through Burp

With the certificate trusted, the emulator's Wi-Fi connection was set to use a manual proxy pointing at the host: proxy hostname 10.0.2.2, port 8080. Saving this setting and then browsing to any HTTPS site confirmed the request appeared in Burp's Proxy → HTTP history, proving that traffic was correctly flowing through the interception point.

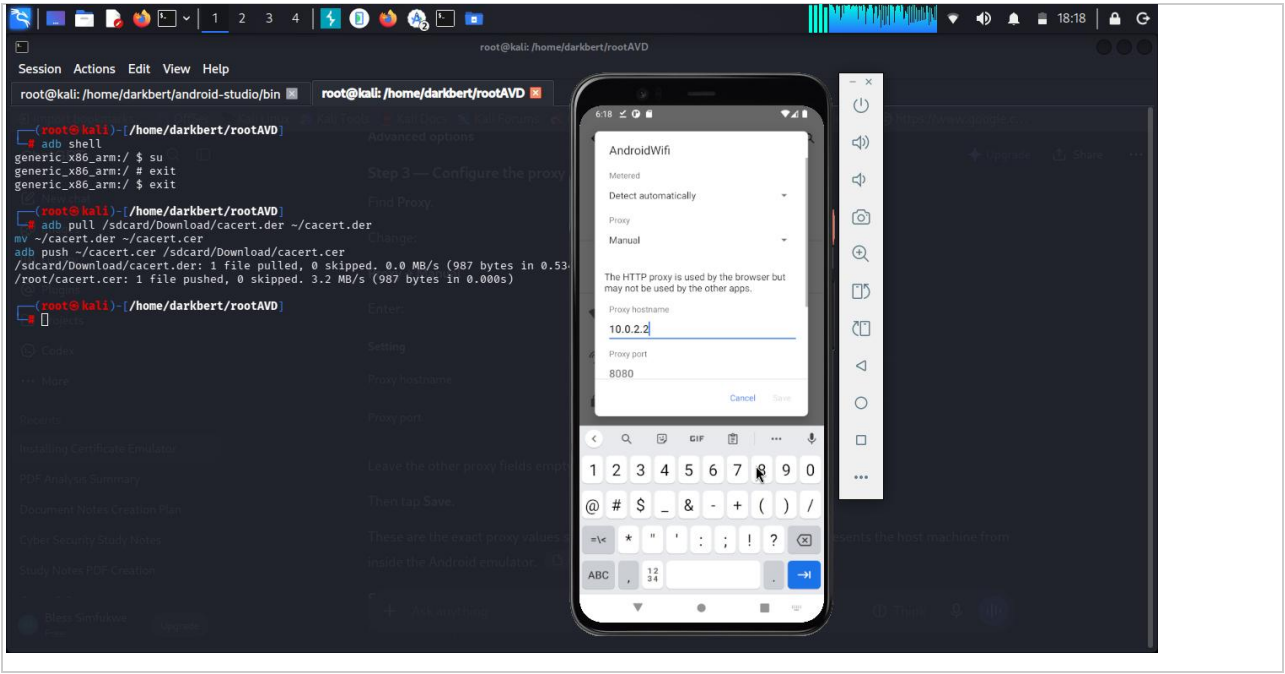


Figure 9. Manually configuring the AVD's Wi-Fi proxy to route all traffic through Burp at 10.0.2.2:8080.

7. Part E – Installing the LSPosed Framework

SSL Killer, the module used later to defeat certificate pinning, is an Xposed-style module and therefore needs a hooking framework to run inside app processes. The Zygisk build of LSPosed was downloaded on the host and pushed to the device's Downloads folder:

```
adb push ~/Downloads/LSPosed-*.zip /sdcard/Download/
```

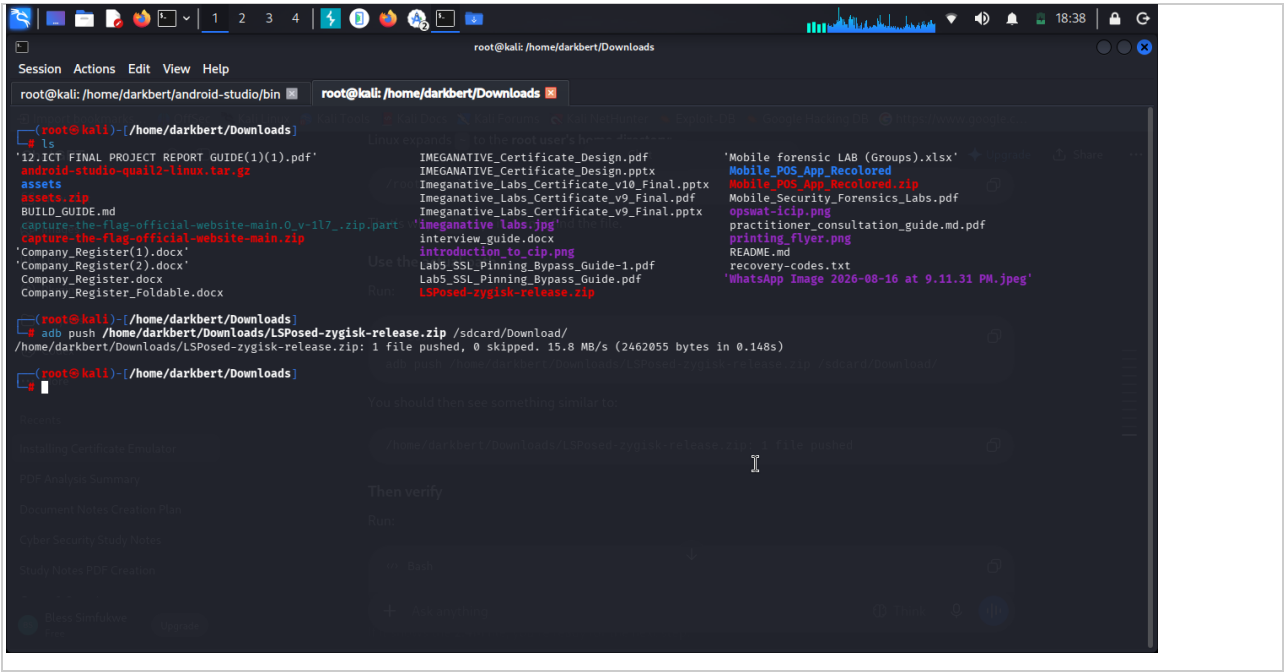


Figure 10. Pushing the LSPosed Zygisk release archive onto the emulator ahead of installation through Magisk.

The module was then installed from within the Magisk app (Modules → Install from storage), which unpacked and verified the LSPosed files, patched the necessary binaries, and finished with a prompt to reboot the device.

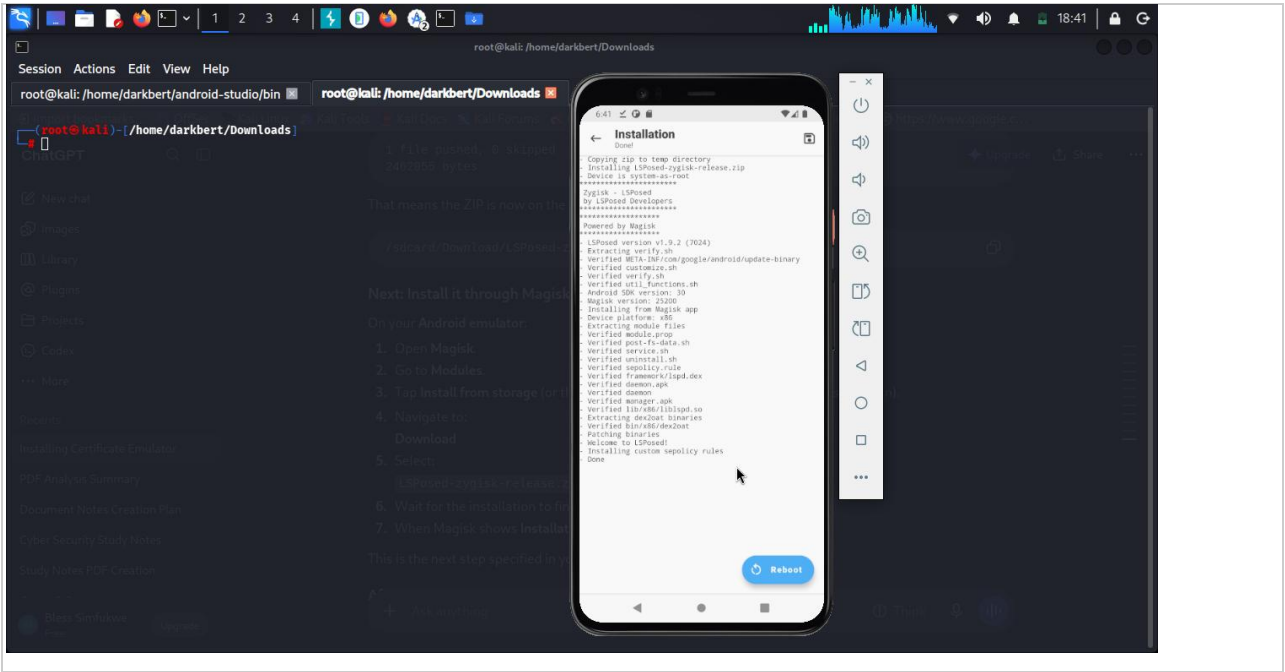


Figure 11. Magisk completing the LSPosed installation, ready for the device to reboot and activate the module.

After the reboot, the new LSPosed Manager app was opened to confirm activation. The home screen reported Framework: Activated (Zygisk mode), version 1.9.2, confirming the hooking framework was live and ready to host modules such as SSL Killer.

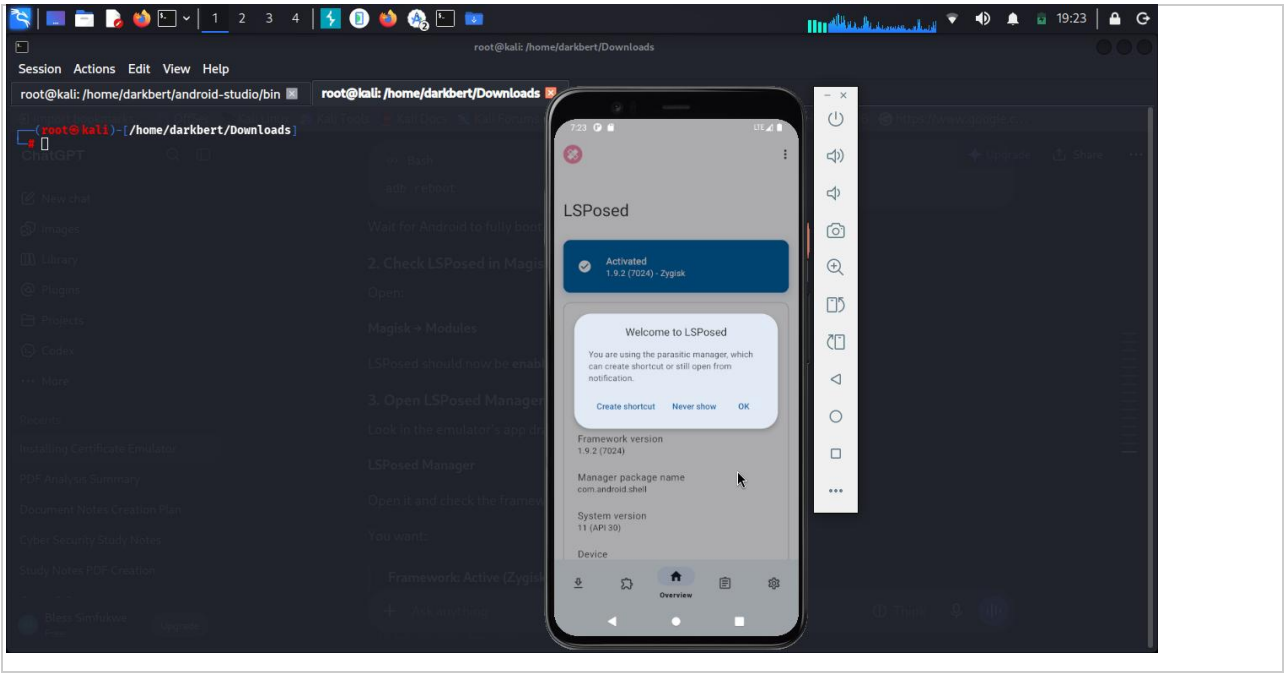


Figure 12. LSPosed Manager confirming the framework is Activated in Zygisk mode after the post-installation reboot.

8. Part F – Installing and Enabling SSL Killer

The SSL Killer APK was installed directly via ADB, then enabled as an LSPosed module. Inside LSPosed Manager, the module was toggled on and the target application was checked in the scrollable app list beneath it, scoping the hook to only the application under test rather than the whole system. The target app's storage and cache were then cleared and the device was rebooted so the hook would apply cleanly to a fresh app process.

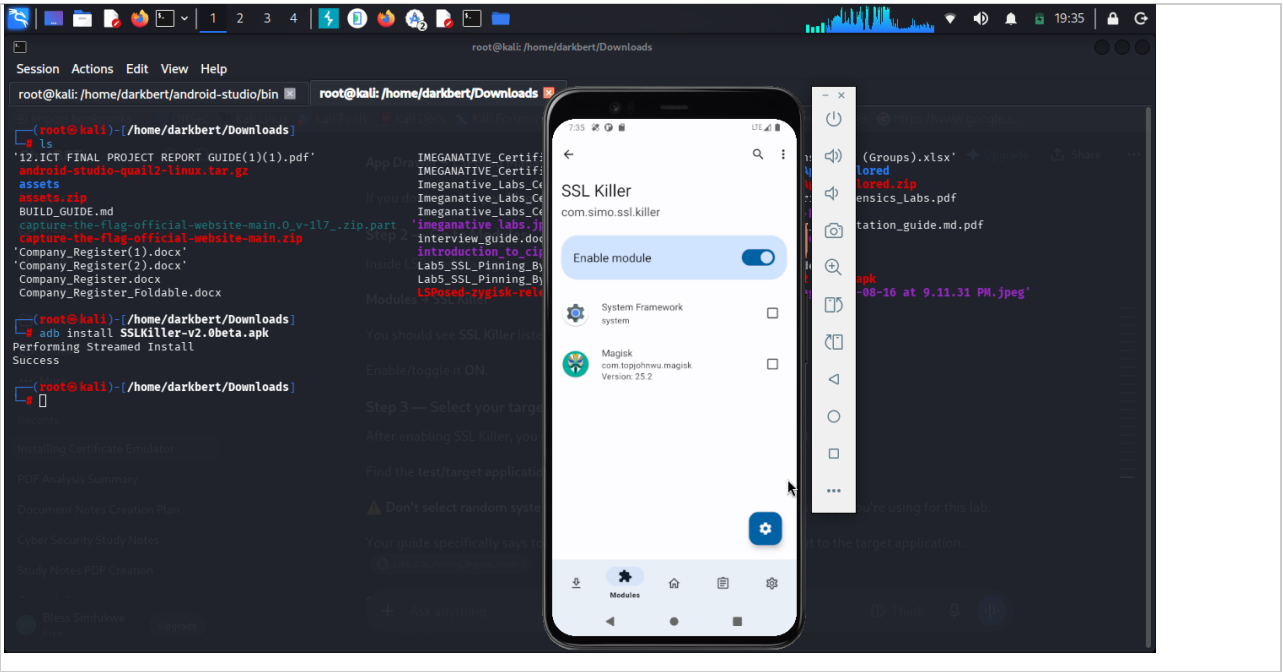


Figure 13. SSL Killer installed via ADB and enabled as an LSPosed module, scoped to the target test application.

9. Part G & H – Intercepting Traffic and Reviewing the Findings

With SSL Killer active, the target application was exercised normally (browsing, login, and similar actions) while Burp's Proxy → HTTP history recorded every request and response. Rather than the connection being silently rejected — which is what certificate pinning is designed to cause — the traffic now passed through Burp for inspection, confirming the bypass was effective.

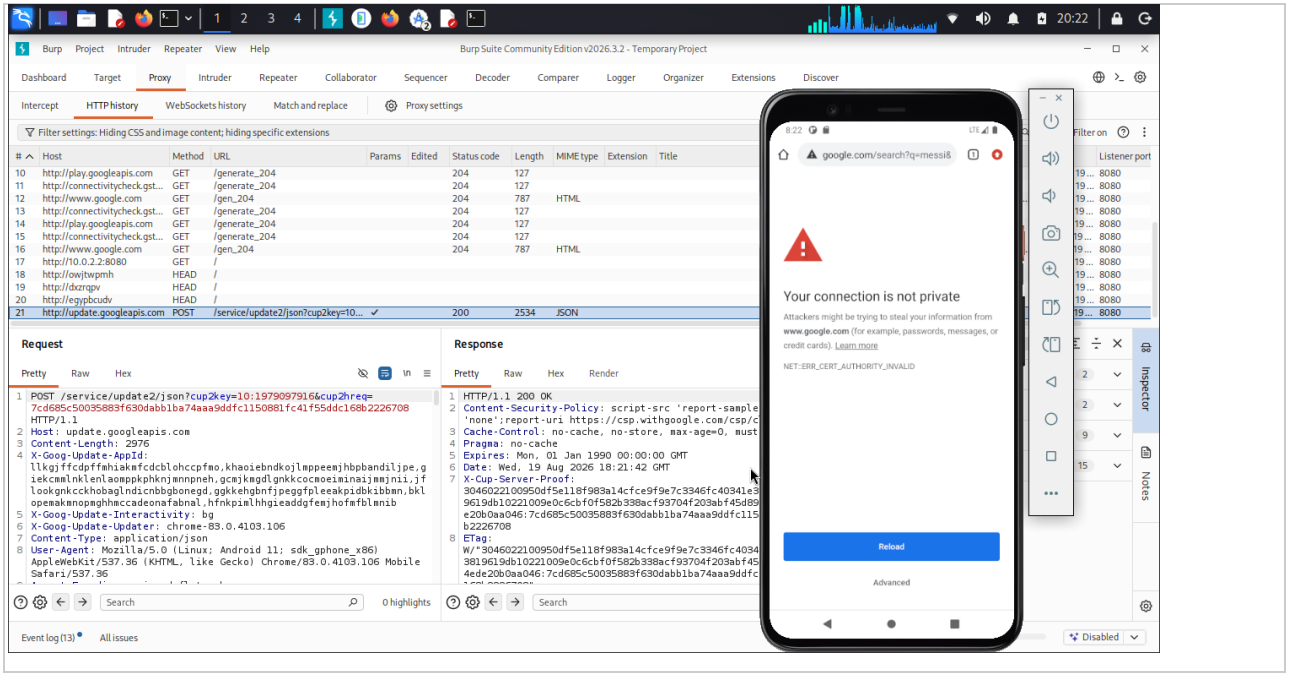


Figure 14. Burp Suite's HTTP history capturing decrypted app traffic after the pinning bypass, alongside a certificate warning surfaced during testing.

The captured history was then reviewed with the specific findings the lab asks for in mind:

- Authentication tokens: checking whether JWT/OAuth tokens were passed only in headers (expected behaviour) or whether any leaked into URL query strings, which would count as a finding.
- Sensitive data exposure: checking whether personally identifiable or financial data was sent with no additional payload-level encryption beyond TLS.
- IDOR (Insecure Direct Object Reference): sending a captured request to Repeater and modifying an account/user ID parameter before resending, to see whether the response returned another user's data.

Each finding identified through this process should be documented with its request/response pair, the affected endpoint, and a short description of the associated risk, exactly as the lab manual specifies. Where the target application returns data belonging to a different user after an ID is altered, this is treated as a reportable IDOR finding.

10. Part I – Fallback Path: Native Pinning via Frida and Objection

The lab guide notes that if SSL Killer fails to intercept traffic and SSL negotiation errors persist, the target app most likely implements pinning at a native (C/C++) level, or uses a non-standard network stack such as Flutter, which Xposed-based hooks cannot reach. For this lab, SSL Killer successfully intercepted the target application's traffic, so this fallback path was not required. For completeness, the documented recovery procedure is summarised below:

- Install Objection: pip install objection
- Confirm frida-server is running on the device: adb shell su -c /data/local/tmp/frida-server & then adb forward tcp:27042 tcp:27042
- Attach to the target app: objection -g com.example.targetapp explore, then run android sslpinning disable inside the Objection shell

As the guide notes, Flutter applications bundle their own BoringSSL implementation and typically need a specialised Frida script rather than the standard Objection bypass, which falls outside the default tooling scope of this lab.

11. Conclusion

Working through this lab end-to-end from booting and rooting the AVD, through installing Burp's CA certificate and routing traffic through the proxy, to standing up LSPosed and using SSL Killer to defeat the target application's certificate pinning gave a practical, hands-on understanding of both how mobile TLS protections work and how they can fail once an attacker (or, in this authorised context, a tester) gains root on the device. Being able to read the resulting HTTP history in Burp turned an otherwise opaque, encrypted app into something fully inspectable, which is exactly the vantage point needed to look for issues like tokens leaking into URLs, unencrypted sensitive data, and IDOR vulnerabilities. The exercise reinforced why certificate pinning alone is not a complete defence on a device the attacker fully controls, and why layered protections server-side authorisation checks in particular matter just as much as pinning itself.

All activity documented in this report was performed strictly within the isolated AVD environment and sample/test application authorised by the lab manual.